



Anomaly Detection for Critical Space Systems

Machine Learning Project Report

Youssouf Farrage
Mathilde Haven
OCC2

12/12/2025

Table of Contents

Links to Datasets	2
1 Business Scope	2
1.1 Problem Definition	2
1.2 Proposed Architecture	2
2 Problem Formalization and Methods	3
2.1 Algorithm Description	3
2.2 Decision-Level Late Fusion Strategy	3
2.3 Limitations	4
2.3.1 System-Level Integration Constraints	4
2.3.2 Network Dataset Limitations: CICIDS2017	4
3 Methodology	4
3.1 Data Description and Exploration	4
3.1.1 Space Segment Dataset: OPSSAT-AD	4
3.1.2 Industrial Segment Dataset: TON_IoT (Modbus)	4
3.1.3 Network Datasets: CICIDS2017	5
3.2 Data Challenges	6
3.2.1 OPSSAT-AD and TON_IoT (Modbus)	6
3.2.2 CICIDS2017	7
3.3 Data Splitting	8
3.4 Algorithm Implementation and Hyperparameters	8
3.4.1 OPSSAT-AD and TON_IoT (Modbus)	8
3.4.2 CICIDS2017	8
3.5 Decision-Level Fusion and System Integration	9
3.5.1 Synthetic Data Generation for System Validation	9
3.5.2 Rule-Based Late Fusion Logic	10
4 Results	10
4.1 Metrics Strategy	10
4.2 Space Segment Results (OPSSAT)	10
4.3 Industrial Segment Results (Modbus)	11
4.4 Network Segment Results (CICIDS2017)	11
4.4.1 Overall Performance	11
4.4.2 Analysis of Ensemble Methods	13
4.4.3 Performance vs Computation Time Trade-off	14
4.4.4 Generalization Validation	14
4.5 System-Level Validation: Multi-Modal Security Fusion	14
5 Discussion and Conclusion	14
References	15

Links to Datasets

- Network Intrusion Dataset (CICIDS2017)
- TON_IoT Datasets
- Satellite Telemetry Data (OPSSAT-AD)

1 Business Scope

1.1 Problem Definition

The space industry is transitioning toward "NewSpace", multiplying interconnected constellations. This evolution exposes missions to cyber threats that are no longer limited to traditional computer networks. An attack can directly target satellite telemetry (to deorbit or damage it) or ground physical infrastructures (to cut communication).

In modern space systems, satellites and ground stations are increasingly exposed to threats such as:

- Spoofing or jamming of communication channels
- Malware injection into ground stations or satellite software
- Unauthorized access to command or control systems

The objective of this project is to design a defense-in-depth system capable of detecting anomalies on these specific layers, where traditional antivirus solutions are inoperative. This work directly aligns with our specialization in Data Science applied to Cybersecurity of cyber-physical systems (CPS), and specialization in aerospace. Additionally, we both have a deep interest in aerospace.

1.2 Proposed Architecture

To address the complexity of a space mission, we designed a modular "Control Center" architecture. Rather than attempting an impossible fusion of heterogeneous data, we developed three "specialist" models:

- **Spatial Specialist:** Monitors the internal health of the satellite (telemetry)
- **Industrial Specialist (OT):** Monitors ground station automation (Modbus)
- **Network Specialist:** Monitors network flows to detect DoS/DDoS attacks (CICIDS2017)

To ensure coherent operation across these independent specialists, we introduce a centralized supervisory component referred to as *Mission Control*. This system-level layer is responsible for orchestrating inference, correlating anomaly detections across domains, and producing a unified security assessment.

Mission Control does not perform feature-level fusion or model training. Instead, it operates at the decision level, applying deterministic correlation rules inspired by operational risk analysis. This design choice enables explainability, robustness, and alignment with the constraints of mission-critical space systems.

2 Problem Formalization and Methods

2.1 Algorithm Description

We address a Supervised Multi-class Classification problem:

- For OPSSAT and Modbus: Binary classification (0 = Normal, 1 = Anomaly)
- For CICIDS2017: Multi-class classification (5 classes: BENIGN, DoS Hulk, DoS GoldenEye, DoS Slowloris, DoS Slowhttptest)

We selected a range of algorithms from the most interpretable to the most performant:

- **Decision Tree:** Chosen as baseline for its ability to provide clear rules (explainability)
- **SVM:** Chosen for its ability to find complex decision boundaries in high-dimensional spaces
- **XGBoost:** Chosen for the industrial module as it represents state-of-the-art for structured tabular data
- **Random Forest:** Chosen for its robustness to noise and ability to capture non-linear relationships
- **Gradient Boosting:** Chosen for its ability to sequentially correct errors from previous trees
- **Logistic Regression:** Used to establish a fast, simple, and interpretable baseline
- **Ensemble Methods (Bagging & Voting):** Used to reduce variance and stabilize predictions
- **Stacking:** Uses a meta-model (Logistic Regression) on base model predictions

2.2 Decision-Level Late Fusion Strategy

In addition to the individual supervised learning models, we introduce a global decision mechanism based on a **Late Fusion** strategy. Rather than merging heterogeneous data at the feature level, each subsystem operates independently and produces its own inference result. These local decisions are then aggregated by a centralized fusion engine.

This approach is motivated by the strong heterogeneity of the data sources involved in a space mission. Fusing these data at an early stage would be both technically incoherent and detrimental to model performance.

The Late Fusion mechanism operates at the *decision level*. Each specialist model outputs a binary anomaly flag (normal vs. anomalous) for its respective domain. A rule-based correlation engine then evaluates combinations of these alerts to infer higher-level security situations. For example, the simultaneous detection of a network intrusion and an industrial protocol anomaly is interpreted as a coordinated ground station attack, while an isolated satellite telemetry anomaly is treated as a localized onboard issue.

This fusion layer does not rely on additional machine learning models. Instead, it implements deterministic security rules inspired by operational risk analysis. The Late Fusion engine thus acts as a supervisory layer, transforming low-level anomaly detections into actionable security decisions.

2.3 Limitations

The major constraint identified from the start is data heterogeneity. It is technically incoherent to merge volts/temperatures (Satellite) and industrial command registers (Modbus) into the same matrix. This justifies our "Expert Models" approach with separate modules.

2.3.1 System-Level Integration Constraints

A deliberate limitation of this work is the absence of a unified end-to-end learning model across all subsystems. While it would be theoretically possible to train a global classifier on concatenated outputs, such an approach would introduce additional opacity and significantly reduce explainability.

In mission-critical aerospace and industrial systems, security mechanisms must remain interpretable and predictable. For this reason, the global decision process is implemented as a deterministic rule-based fusion layer rather than a learned model. This choice avoids cascading errors and simplifies validation.

2.3.2 Network Dataset Limitations: CICIDS2017

- **Class Imbalance:** BENIGN class represents 79% of data; models may be biased toward majority class
- **Strategic Attack Selection:** We deliberately selected Monday (BENIGN) and Wednesday (DoS variants) for a controlled scope on denial-of-service attacks
- **Temporal Coverage:** Data collected during specific time periods may not represent all attack patterns
- **Sampling:** Used 50,000 samples instead of full 700k+ dataset for computational efficiency
- **Grid Search Scope:** Limited hyperparameter combinations tested

3 Methodology

3.1 Data Description and Exploration

3.1.1 Space Segment Dataset: OPSSAT-AD

We chose this dataset because it comes from a real ESA (European Space Agency) mission, guaranteeing business relevance. It contains statistical features (mean, variance, skewness) extracted from telemetry segments.

Furthermore, we deliberately excluded the raw data (segments.csv) to use the preprocessed version (dataset.csv). This allowed us to focus effort on model optimization (GridSearch, Ensembling) rather than on low-level Data Engineering.

3.1.2 Industrial Segment Dataset: TON_IoT (Modbus)

For the ground station, we explored the TON_IoT dataset suite.

- **Strategic Selection:** After analyzing the directory structure, we selected the Train_Test_datasets folder rather than Raw_datasets or GroundTruth because this folder provides cleaned and labeled data ready for ML, ensuring reproducibility of our results against scientific benchmarks.

- **Protocol Choice:** In the IoT subfolder, we chose Modbus.csv. Unlike "Fridge" or "GPS" datasets, the Modbus protocol is the industrial standard used to control antenna motors and ground station power supply. This is the most realistic choice for our use case.

3.1.3 Network Datasets: CICIDS2017

In this project, we used two specific subsets of the CICIDS2017 dataset: `Monday-WorkingHours.pcap_ISCX.csv` and `Wednesday-WorkingHours.pcap_ISCX.csv`. These two files were chosen because they provide a clean and balanced foundation for supervised intrusion detection.

The Monday-WorkingHours dataset contains exclusively benign traffic, recorded during normal office activity. This includes legitimate HTTP/HTTPS browsing, email exchanges, DNS queries, chat applications, and regular user behavior. Because no attacks are present, this dataset serves as a reliable baseline to learn what "normal" network telemetry looks like.

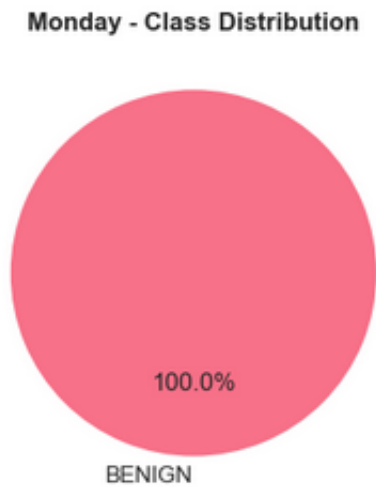


Figure 1: Class Distribution - Monday Dataset (100% BENIGN traffic)

The Wednesday-WorkingHours dataset, in contrast, includes a mixture of benign traffic and several major DoS attacks such as Hulk, SlowHTTPTest, Slowloris, and GoldenEye. These attacks are widely used in cybersecurity research because they represent realistic and diverse denial-of-service strategies.

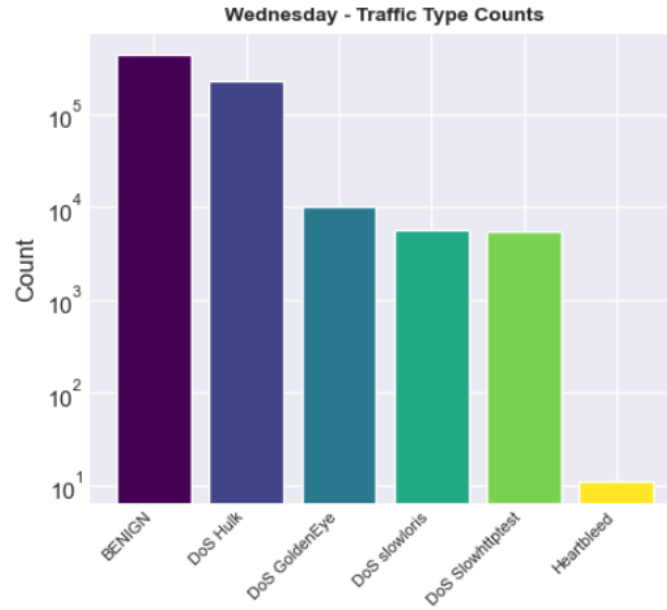


Figure 2: Traffic Type Count - Wednesday Dataset

By combining these three datasets, we can build a complete model that considers attacks from three perspectives: network, system, and satellite telemetry. This multi-dimensional approach makes detection more robust and allows us to apply machine learning methods to different types of data.

3.2 Data Challenges

3.2.1 OPSSAT-AD and TON_IoT (Modbus)

- **Missing Values:** The OPSSAT-AD and TON_IoT (Modbus) datasets are clean (no NaN)
- **Outliers & Scaling:** Since sensor data has very different scales, we applied StandardScaler (mean=0, std=1) to allow SVMs to converge
- **Temporal Bias (Modbus):** We identified a critical risk of overfitting on the date and time columns. We removed them to force the model to learn behaviors (command patterns) rather than attack schedules

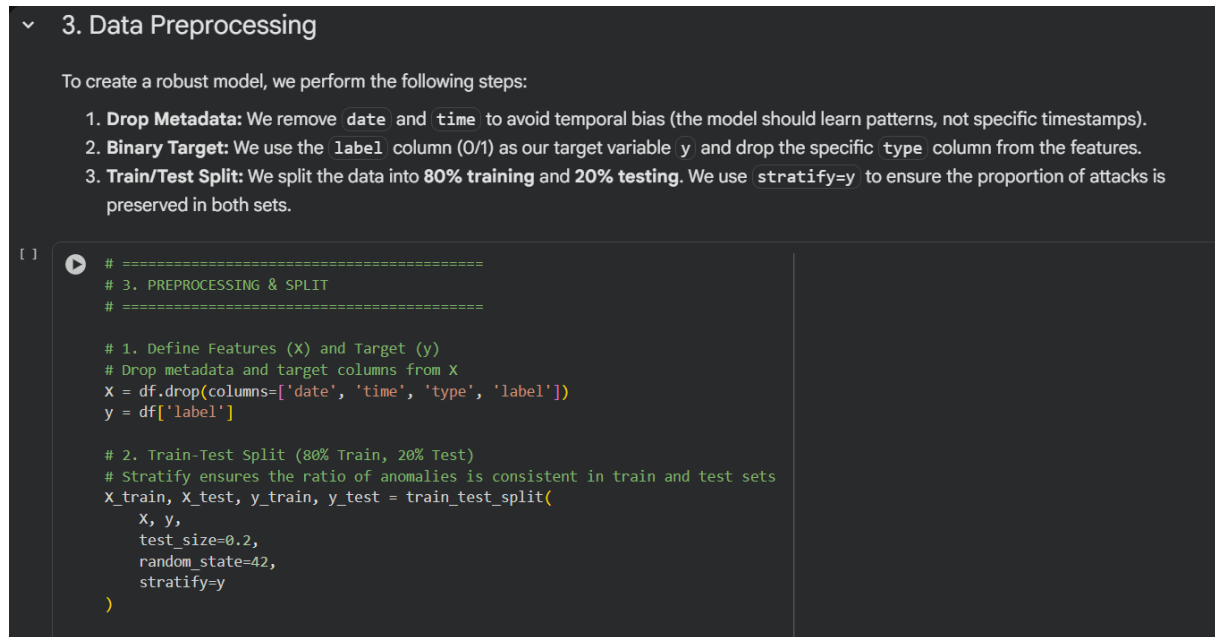


Figure 3: Data Preprocessing

3.2.2 CICIDS2017

- **Missing Values:** After combining the datasets, we replaced inf values with NaN, then removed rows with NaN
- **Imbalance ratio:** 184:1 (BENIGN vs. smallest class DoS Slowhttptest). We preserved the natural imbalance without applying SMOTE or undersampling, because ensemble methods intrinsically handle imbalance
- **Outliers:** Outliers are often attack signatures; removing them would remove attack patterns. Using StandardScaler reduces the impact of scale differences without removing outliers

```
# Clean data
data = data.replace([np.inf, -np.inf], np.nan)
data = data.dropna()

# Encode Labels
le = LabelEncoder()
data['Label_encoded'] = le.fit_transform(data['Label'])
num_classes = len(le.classes_)

# Separate features and target
X = data.drop(['Label', 'Label_encoded'], axis=1)
y = data['Label_encoded']

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Normalization
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Figure 4: Data Preprocessing

3.3 Data Splitting

For OPSSAT and TON_IoT, we rigorously respected the Train/Test splits provided by the authors (train column for OPSSAT, separate files for TON_IoT).

For CICIDS2017, we combined Monday (BENIGN only) and Wednesday (BENIGN + DoS attacks) then performed a stratified random 80/20 split (`train_test_split` with `stratify=y`, `random_state=42`). We made this choice because the two days are independent (no temporal continuity between Monday and Wednesday).

3.4 Algorithm Implementation and Hyperparameters

We used GridSearchCV for rigorous optimization:

3.4.1 OPSSAT-AD and TON_IoT (Modbus)

- **Decision Tree (OPSSAT)**
 - Optimal configuration: `criterion='gini'`, `max_depth=3`, `min_samples_split=2`, `min_samples_leaf=1`
 - GridSearch revealed that a shallow tree (depth 3) was sufficient. It achieved excellent Precision (98.8%) but struggled with Recall (0.743), meaning it was too conservative and missed subtle anomalies.
- **SVM (OPSSAT)**
 - Optimal configuration: `kernel='linear'`, `C=0.1`, `gamma='scale'`, `probability=True`
 - The linear kernel proved effective for this feature space. Enabling `probability=True` allowed probability averaging. This model better balanced Precision and Recall than the Tree (F1: 0.856).
- **VotingClassifier (OPSSAT)**
 - Configuration: `voting='soft'`, `estimators=[('bag_svm', bag_svm), ('bag_dt', bag_dt)]`
 - We combined Bagged versions of the best SVM and Tree using Soft Voting. This gave the best global performance (F1 = 0.8670) and the lowest False Negative rate (25 missed attacks vs 29 for the DT). We accepted a slightly higher inference time (0.088s vs 0.059s for SVM) to maximize mission safety (Recall).
- **XGBoost (Industrial Modbus)**
 - Optimal configuration: `n_estimators=100`, `learning_rate=0.1`, `max_depth=5`, `eval_metric='logloss'`
 - For the larger Modbus dataset (30k rows), we used Gradient Boosting as a standalone strong learner. A moderate depth of 5 prevented overfitting on sensor noise. Unlike the OPSSAT module, no external voting was needed because XGBoost naturally corrects errors iteratively, achieving >95% accuracy with fast inference.

3.4.2 CICIDS2017

- **Random Forest:**
 - Optimal configuration: `max_depth=None`, `min_samples_split=2`, `n_estimators=100`
 - Maximum depth (None) to capture complex DoS patterns, moderate number of trees (100) to balance performance and computation time (114.26s)

- **Decision Tree:**

- Optimal configuration: `criterion='entropy', max_depth=20, min_samples_leaf=1, min_samples_split=2`
- Entropy criterion improves separation of minority classes (DoS variants), depth of 20 avoids overfitting while capturing attack signatures

- **Gradient Boosting:**

- Optimal configuration: `learning_rate=0.1, max_depth=5, n_estimators=50`
- Deliberately lightweight configuration to reduce computation time (772s), limited depth for regularization

- **XGBoost:**

- Optimal configuration: `learning_rate=0.2, max_depth=7, n_estimators=200, subsample=1.0`
- High learning rate (0.2) for fast convergence, 200 estimators for maximum performance, `subsample=1.0` because dataset already sampled

- **Logistic Regression:**

- Optimal configuration: `C=10, penalty='l2', max_iter=1000`
- Linear baseline to compare with complex models, `C=10` for light regularization

- **Ensemble Methods:**

- `VotingClassifier`: Combines `RandomForest`, `DecisionTree`, `Gradient Boosting`, and `XGBoost` with `voting='hard'` (majority vote) and `voting='soft'` (probability averaging). Soft voting exploits prediction probabilities for more nuanced decision-making
- `BaggingClassifier`: 5 estimators based on optimized `Random Forest`, reduces variance through bootstrap aggregating
- `StackingClassifier`: Uses `Random Forest`, `DecisionTree`, `Gradient Boosting`, and `XGBoost` as base models, with `Logistic Regression (max_iter=1000)` as meta-learner. Stacking learns the optimal combination of base model predictions

3.5 Decision-Level Fusion and System Integration

To complement the independent specialist models, we implemented a centralized system-level integration module referred to as *Mission Control*. This component is responsible for orchestrating inference across the Space, Industrial, and Network subsystems and for correlating their outputs into a unified security assessment.

The *Mission Control* module operates exclusively at runtime and does not perform any model training. Instead, it loads pre-trained model artifacts generated during the experimentation phase and applies them to incoming data streams. This design reflects realistic operational constraints, where retraining is infeasible during a mission and models must be validated prior to deployment.

3.5.1 Synthetic Data Generation for System Validation

In the absence of live telemetry and network streams, the *Mission Control* module relies on synthetic data generation to validate the end-to-end behavior of the system. These synthetic vectors are generated to strictly match the dimensionality and feature schema of the original

training datasets used for each subsystem.

Distinct simulation scenarios are defined to emulate nominal operation and targeted attacks on specific layers, including satellite telemetry anomalies, network denial-of-service patterns, and industrial protocol abuse. This approach allows functional validation of the integration logic without biasing results with unseen real-world data.

3.5.2 Rule-Based Late Fusion Logic

Each specialist model produces an independent anomaly decision for its respective domain. These decisions are then processed by a Late Fusion engine that applies deterministic correlation rules to identify higher-level attack scenarios.

The fusion logic explicitly models domain knowledge: isolated anomalies are treated as localized incidents, while simultaneous alerts across multiple subsystems are interpreted as coordinated attacks. This hierarchical reasoning enables the system to escalate threat levels only when cross-domain evidence is present, reducing the risk of false critical alarms.

4 Results

4.1 Metrics Strategy

In space, losing a satellite is unacceptable. We therefore chose to focus our energy on a few metrics that seemed most important to us:

- **Priority Metric - Recall:** We must minimize False Negatives (missed attacks)
- **F1-Score:** To maintain balance and not classify everything as "attack"
- **Inference Time:** Crucial for resource-limited embedded systems

4.2 Space Segment Results (OPSSAT)

We compared base models with ensembles. The VotingClassifier achieves the best overall performance (F1-score ≈ 0.867).

Engineering Trade-offs:

Engineering is the art of compromise. Like designing a fighter jet (weight vs speed), we had to arbitrate between performance and computational cost.

Model	F1-Score	Inference Time (s)
Bagging SVM	0.8529	0.0597
Voting Classifier	0.8670	0.0882

Table 1: OPSSAT Performance Comparison

Analysis: Bagging SVM is 30% faster. However, for satellite security, the reliability gain (+1.4% F1 and fewer False Negatives) justifies this time cost. We therefore selected the VotingClassifier.

```

... Bagging Decision Tree - Accuracy: 0.9433
Bagging Decision Tree - Precision: 0.9882
Bagging Decision Tree - Recall: 0.7434
Bagging Decision Tree - F1-score: 0.8485
Bagging Decision Tree - Inference time (s): 0.017042
-----
Bagging SVM - Accuracy: 0.9433
Bagging SVM - Precision: 0.9560
Bagging SVM - Recall: 0.7699
Bagging SVM - F1-score: 0.8529
Bagging SVM - Inference time (s): 0.059705
-----
Voting (Bagging SVM + Bagging DT) - Accuracy: 0.9490
Voting (Bagging SVM + Bagging DT) - Precision: 0.9778
Voting (Bagging SVM + Bagging DT) - Recall: 0.7788
Voting (Bagging SVM + Bagging DT) - F1-score: 0.8670
Voting (Bagging SVM + Bagging DT) - Inference time (s): 0.088226
-----

```

Figure 5: Space Segment Results (OPSSAT)

4.3 Industrial Segment Results (Modbus)

For this part, we did not use VotingClassifier but the XGBoost algorithm. Because unlike a simple tree, XGBoost is intrinsically an ensemble model (Gradient Boosting). Adding a Voting layer on top would have increased computation without significant gain. We were right as the model achieves Accuracy > 95% on the test set, proving it can distinguish malicious command injections from normal maintenance operations.

```

... Evaluating the best model on Test set...

--- Inference Speed ---
Total prediction time : 0.1023 seconds
Latency per sample   : 0.00001644 seconds

--- Performance Metrics ---
Accuracy : 0.9208
Precision : 0.9265
Recall : 0.9199 <-- Critical metric
F1-Score : 0.9232

```

Figure 6: Industrial Segment Results (Modbus)

Overfitting Analysis:

Training and test scores remain close (< 5% gap) on both modules, validating the effectiveness of our regularization (GridSearch).

4.4 Network Segment Results (CICIDS2017)

4.4.1 Overall Performance

For the network module, we obtained very good results with a dataset of 50,000 samples representing 5 classes (BENIGN and 4 types of DoS attacks). The Stacking model achieves the best overall performance with 99.96% accuracy, followed very closely by XGBoost (99.95%) and the Voting Classifiers.

Model	Accuracy	Precision	Recall	F1-Score	Time (s)
Stacking	0.9996	0.9996	0.9996	0.9996	454.36
XGBoost	0.9995	0.9995	0.9995	0.9995	213.81
Voting (Hard)	0.9995	0.9995	0.9995	0.9995	-
Voting (Soft)	0.9995	0.9995	0.9995	0.9995	-
Random Forest	0.9993	0.9993	0.9993	0.9993	114.26
Decision Tree	0.9993	0.9993	0.9993	0.9993	75.72
Bagging	0.9989	0.9989	0.9989	0.9989	-
Gradient Boosting	0.9982	0.9982	0.9982	0.9982	772.72
Logistic Regression	0.9947	0.9949	0.9947	0.9947	9.85

Table 2: Results Summary - All Models on CICIDS2017

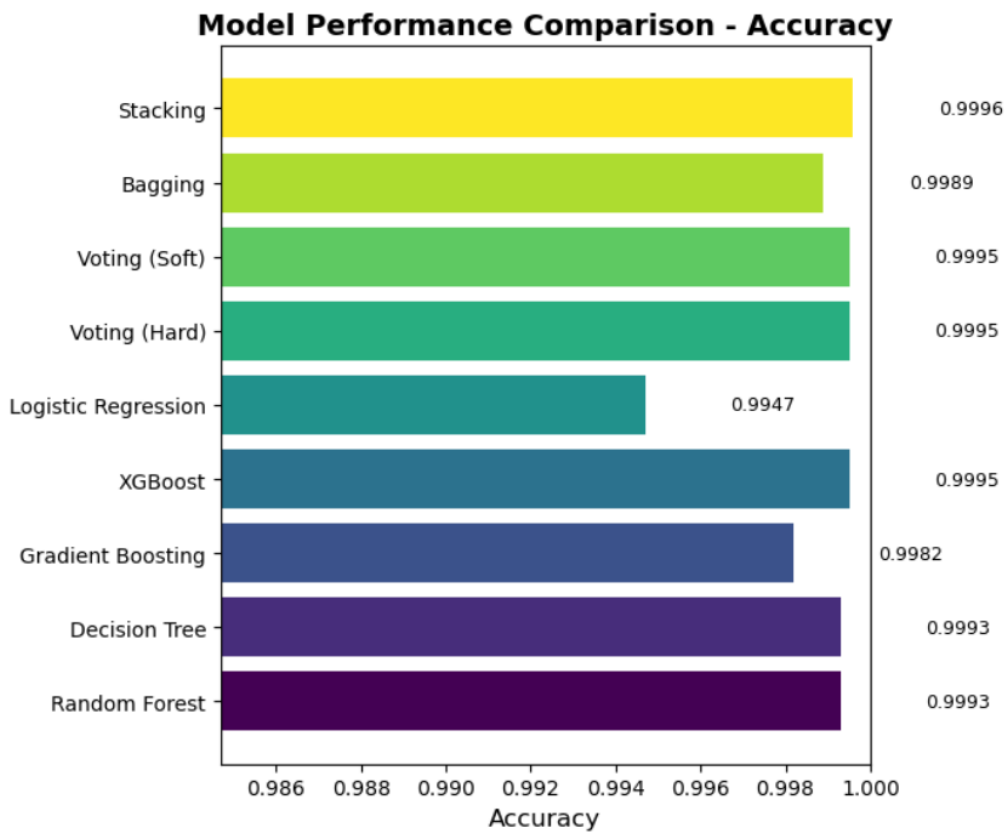


Figure 7: Model Performance Comparison - Accuracy

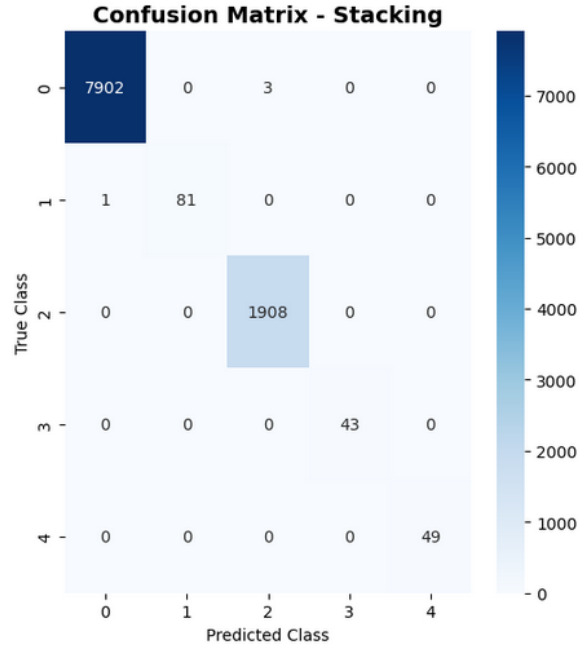


Figure 8: Confusion Matrix - Best Model (Stacking)

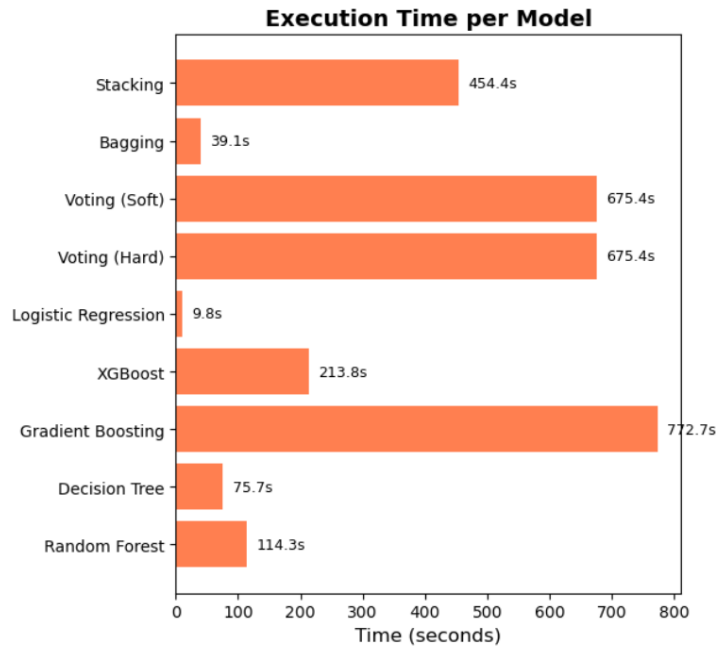


Figure 9: Execution Time per Model

4.4.2 Analysis of Ensemble Methods

Ensemble methods demonstrated clear superiority over individual models:

- **Stacking (99.96%):** Combines RandomForest, DecisionTree, Gradient Boosting, and XGBoost with a meta-learner (Logistic Regression). The diversity of base models allows capturing different perspectives on attack patterns

- **XGBoost (99.95%):** Although already an intrinsic ensemble, XGBoost remains extremely performant and offers the best performance/inference time trade-off (213.81s vs 454.36s for Stacking)
- **Voting Classifiers (99.95%):** Hard and Soft voting achieve the same performance as XGBoost, confirming the effectiveness of combining heterogeneous models

4.4.3 Performance vs Computation Time Trade-off

In accordance with our engineering compromise philosophy:

Model	Accuracy	Training Time (s)	Perf/Time Ratio
Stacking	99.96%	454.36	0.0022
XGBoost	99.95%	213.81	0.0047
Random Forest	99.93%	114.26	0.0087

Table 3: Performance vs Computation Time Trade-off

Analysis: Stacking offers a 0.01% accuracy gain over XGBoost but takes $2.1\times$ longer. For a real-time network detection application where every millisecond counts, XGBoost represents the best operational choice: it offers 99.95% accuracy with inference time reduced by half.

4.4.4 Generalization Validation

Cross-validation (CV) scores remain very close to test scores:

- XGBoost: CV=0.9991 vs Test=0.9995 ($\Delta=0.04\%$)
- Random Forest: CV=0.9982 vs Test=0.9993 ($\Delta=0.11\%$)
- Decision Tree: CV=0.9983 vs Test=0.9993 ($\Delta=0.10\%$)

This consistency (gap $< 0.5\%$) confirms that our models generalize well and are not overfitting, validating our GridSearch strategy with cv=2.

4.5 System-Level Validation: Multi-Modal Security Fusion

Beyond individual model performance, we evaluated the behavior of the complete system through scenario-based simulations. Four operational contexts were considered: nominal operation, isolated network attack, isolated satellite anomaly, and a coordinated ground station attack combining network and industrial intrusions.

The system successfully identified the coordinated ground station attack as the highest criticality scenario. This confirms the effectiveness of the Late Fusion strategy in correlating weak signals across independent subsystems. Importantly, even when individual models exhibit sensitivity or noise, the fusion layer maintains robust decision-making by relying on cross-domain confirmation.

5 Discussion and Conclusion

We conducted this project with a comprehensive "Data Scientist" approach, from critical selection of data sources to arbitration of performance trade-offs.

We delivered three functional modules:

- **Spatial Module (OPSSAT):** VotingClassifier optimized for reliability (Recall)

- **Industrial Module (TON_IoT Modbus):** XGBoost, capable of securing ground SCADA protocols
- **Network Module (CICIDS2017):** XGBoost with 99.95% Accuracy (optimal performance/time choice)

However, securing a space mission cannot rely on isolated detectors alone. In operational environments, threats often span multiple physical and logical layers, requiring correlated analysis across heterogeneous systems. To address this challenge, we extended our work beyond individual model performance and designed a system-level integration layer capable of aggregating and interpreting these independent detections.

References

- OPSSAT-AD Dataset: Kaggle / European Space Agency
- Moustafa, N. (2021). TON_IoT Datasets: UNSW Canberra
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System
- CICIDS2017 Dataset: Kaggle
- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. ICISSP
- Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*, 5(2), 241-259